Hannah D'Cruz
Dr. Ming Li
CS 2123
5 Nov 2025

Huffman Encoding and Decoding

This project was to read a text file, create a Huffman trie for it, compress the file using Huffman encoding into a smaller binary file, and then decode the binary file back into a text file to confirm they are the same.

I implemented this project by first reading each character in the text file sequentially and making a HashMap to store the number of occurrences of each character, visible and non-visible. This is important to make the Huffman trie, because more frequent letters are weighted higher to build the tree. I put each of the characters from the HashMap into their own nodes to store the character and the weight. Because I eventually wanted the total weight of multiple individual characters while keeping track of how they were combined, each node stores a nested ArrayList of characters. Each node starts with an ArrayList with one character and the weight of that character from the HashMap in the previous step. I put all these nodes into a PriorityQueue so I can always get the next smallest value, and then poll the queue repeatedly to get the two lowest weighted characters, the ones that occur least often. I combined those two nodes into a new node with both ArrayLists into a new ArrayList and the sum of the weights stored as the weight, and inserted this back into the queue. The first item in the ArrayList is therefore the left child, and the second item in the ArrayList is the right child. I repeated this process until there was only one node in the queue. This node holds the nested ArrayList of all the characters that represents the Huffman trie. Once I had the trie, I had to read it to get a String version of the encodings for each character. To do this, I wrote a recursive method that calls itself on each half of the trie until the length of the ArrayList is one, simultaneously keeping track of whether it went left or right by building up a String with zeros and ones. Once there is only one element in the next lowest ArrayList, we have found a leaf node, so I add the character and the resulting String of Huffman encoding to a HashMap. Then it works its way back up to the previous level, removes the last level's zero or one from the String, and takes the other branch. I chose a HashMap to store the characters and encodings because it is important to be able to convert between them quickly due to the number of characters in a book. Since HashMaps are unidirectional, I also made a reverse HashMap to go from encodings to characters to help with decoding later.

Once we have the encodings for each character, we can read the file again to actually encode it. This time, instead of counting the occurrences of each character, we use the HashMap to get the equivalent encoding for each character, and add that String to all the previous characters' encodings to make one enormous String of the entire file encoded. Once we have this string, we need to store it in a binary file. To do this, I take each substring of eight bits and convert it to the equivalent integer value, and then cast that to a byte. I can't go straight to storing it as a byte because Java calls all bytes signed, but it was parsing the String into an unsigned value and therefore threw an error that the String value was out of range to fit into a byte. By converting it to an integer and then casting it as a byte, it simply takes the lowest bits of the integer variable and stores them, so then it accepts that the values fit. Casting from an integer to a byte will never result in a loss of precision because we started with eight bits in the String anyway, the integer just sign-extended those eight bits to fill its whole variable. Once we have written the equivalent byte value into the binary file, we remove those eight bits from the giant String and repeat the process until we reach the

end of the String. If the String is not an exact multiple of eight, the last byte to write into the file might not have eight bits from the encoding, so we append zeros to the end as needed to fill the last byte. Once that is done, we have a complete binary Huffman compression of the text file. It seemed unnecessarily onerous to include which book number was being compressed in the name of the .bin file, so I just gave it a generic name. We can then compare the size of the original and compressed files by calculating the compression ratios.

After we created the binary file, we had to decode it again. Similar to how I encoded the file, I read each byte of the binary file and added it to a really long String of all the bits in the file. I converted from byte to unsigned integer and then from integer into base-2 String because there is no way to go directly from a byte in base-2 to a String. Since the integer is unsigned, there is no problem with converting between the two, because the integer will sign-extend with zeros, so the value will be the same as the eight bits we originally put in there. Converting from a base-2 integer to a String only makes as many bits as needed for the value, it does not automatically make the result 8 bits, so I had to manually prepend zeros to the result to fill the whole byte because when we compressed it we put each set of eight bits, whether they were zeros or ones, into a full byte. Once I had the very long string of all the converted bits again, I could read them to get characters back. Because the Huffman trie makes sure all the prefixes are different, I could start with just one bit and gradually read more bits until I found a String of bits that matched one of the keys in my reverse HashMap of encodings to characters. Once I converted those bits into their character, I ignored those and moved onto the next ones. Because I added extra zeros to the end of the original string to fill the last byte, I was careful to only convert as many characters as we had counted in the original text. Once I got enough characters decoded to match the original file, I could compare the resulting file with the original file to make sure they were the same.

Book	Compression Ratio
book1.txt	0.6035795346
book2.txt	0.5949025739
book3.txt	0.5521984459
book4.txt	0.5896459697
book5.txt	0.5874337034
book6.txt	0.6010194102

The compression ratio for all of the books was about 60% (see table). This makes sense because the common symbols are compressed to be 3 or 4 bits, about half the length of the 8 bits they were originally stored with, but the less common symbols are actually longer than 8 bits, so the average compression amount makes the total about 60% of the original. A higher compression ratio means the compression was a bit less successful, probably due to the presence of more unique characters. I found it interesting that the English texts, books 1, 3, and 6 had both the highest and the lowest compression ratios, while the foreign language texts all have very similar compression ratios. The two Shakespeare works had the highest compression ratios, probably because they are written in verse and all the names are in capital letters, so they have more different characters.

The top five most common symbols in book1.txt (Romeo and Juliet) were “ “ with 24,842 occurrences, “e” with 13,648 occurrences, “t” with 10,237 occurrences, “o” with 9,326 occurrences, and “a” with 8,309 occurrences. These symbols therefore had the shortest Huffman encodings in order to be able to compress the file as much as possible. The Huffman encoding for a space was “110,” the only character with only 3 bits. The other characters’ encodings were all four bits long: “e” corresponded to “1110,” “t” to “1000,” “o” to “0110,” and “a” to “0100.” This makes sense because the purpose of doing the Huffman encoding is to make the file take up less space, so you want the most common characters to not take up very much space. This means putting them higher up in the trie, because each digit of the code corresponds to another level down in the trie.